

1. Chuẩn bị cơ sở dữ liệu (MariaDB)

Chạy script SQL sau trong DBeaver, MySQL Workbench hoặc MariaDB CLI để tạo database `storedb`, 2 bảng `Product`, `ShoppingCart` cùng dữ liệu mẫu (10 sản phẩm có URL ảnh và 20 giỏ hàng):

```
```sql
CREATE DATABASE IF NOT EXISTS storedb CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;
USE storedb;

DROP TABLE IF EXISTS ShoppingCart;
DROP TABLE IF EXISTS Product;

CREATE TABLE Product (
 id INT AUTO_INCREMENT PRIMARY KEY,
 name VARCHAR(255) NOT NULL,
 price DOUBLE NOT NULL,
 description TEXT,
 image_url VARCHAR(500)
);

CREATE TABLE ShoppingCart (
 id INT AUTO_INCREMENT PRIMARY KEY,
 product_id INT NOT NULL,
 customer_name VARCHAR(100) NOT NULL,
 quantity INT NOT NULL,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 CONSTRAINT fk_cart_product FOREIGN KEY (product_id) REFERENCES Product(id)
 ON DELETE CASCADE
);

-- Insert 10 Products
INSERT INTO Product (name, price, description, image_url) VALUES
('iPhone 15 Pro', 999.0, 'Titanium design, A17 Pro chip',
'https://picsum.photos/id/1/400/400'),
('Samsung Galaxy S24', 850.0, 'Galaxy AI, Dynamic AMOLED',
'https://picsum.photos/id/2/400/400'),
('MacBook Air M2', 1199.0, '13.6-inch Liquid Retina Display',
'https://picsum.photos/id/3/400/400'),
('Dell XPS 13', 1050.0, 'Intel Core i7 13th Gen, FHD+', 'https://picsum.photos/id/4/400/400'),
('Sony WH-1000XM5', 399.0, 'Noise Canceling Headphones',
'https://picsum.photos/id/5/400/400'),
('iPad Air 5', 599.0, 'Apple M1 chip, 10.9-inch display', 'https://picsum.photos/id/6/400/400'),
```

```
('Logitech MX Master 3S', 99.0, 'Wireless Performance Mouse',
'https://picsum.photos/id/7/400/400'),
('Keychron K2 V2', 79.0, 'Wireless Mechanical Keyboard',
'https://picsum.photos/id/8/400/400'),
('Apple Watch Series 9', 399.0, 'S9 SiP chip, Always-On Retina',
'https://picsum.photos/id/9/400/400'),
('LG UltraFine 4K', 450.0, '27-inch IPS UHD Display', 'https://picsum.photos/id/10/400/400');
```

-- Insert 20 ShoppingCart records

```
INSERT INTO ShoppingCart (product_id, customer_name, quantity) VALUES
(1, 'Nguyen Van A', 1), (2, 'Tran Thi B', 2), (3, 'Le Van C', 1), (4, 'Pham Thi D', 1),
(5, 'Hoang Van E', 3), (6, 'Nguyen Thi F', 1), (7, 'Vu Van G', 2), (8, 'Dang Thi H', 4),
(9, 'Bui Van I', 1), (10, 'Do Thi K', 2), (1, 'Le Van L', 2), (3, 'Tran Thi M', 1),
(5, 'Nguyen Van N', 1), (2, 'Pham Van O', 3), (4, 'Bui Thi P', 1), (7, 'Hoang Thi Q', 2),
(8, 'Do Van R', 1), (6, 'Nguyen Van S', 2), (10, 'Tran Van T', 1), (9, 'Le Thi U', 1);
```

...

---

## ## 2. Cấu hình Gradle (`build.gradle`)

Apache Tomcat 10 chỉ hỗ trợ Servlet container (Jakarta EE 9/10 namespace `jakarta.\*`). Để chạy Jakarta RESTful Web Services (JAX-RS) và CDI trên Tomcat 10, dự án cần nạp Jersey (REST engine) và Weld Servlet (CDI engine).

Mở file `build.gradle` và cấu hình:

```
```groovy
plugins {
    id 'java'
    id 'war'
}

group = 'vn.fit'
version = '1.0-SNAPSHOT'

repositories {
    mavenCentral()
}

ext {
    jerseyVersion = '3.1.5'
    weldVersion = '5.1.2.Final'
}
```

```

dependencies {
    // Servlet API cho Tomcat 10
    compileOnly 'jakarta.servlet:jakarta.servlet-api:6.0.0'

    // CDI API
    compileOnly 'jakarta.enterprise:jakarta.enterprise.cdi-api:4.0.1'

    // Weld Servlet cho CDI trên Tomcat
    implementation "org.jboss.weld.servlet:weld-servlet-core:${weldVersion}"

    // Jersey REST Engine & JSON Jackson binding
    implementation "org.glassfish.jersey.containers:jersey-container-servlet:${jerseyVersion}"
    implementation "org.glassfish.jersey.inject:jersey-cdi2-se:${jerseyVersion}"
    implementation "org.glassfish.jersey.media:jersey-media-json-jackson:${jerseyVersion}"

    // MariaDB Connector
    implementation 'org.mariadb.jdbc:mariadb-java-client:3.3.3'

    testImplementation 'org.junit.jupiter:junit-jupiter-api:5.10.0'
    testRuntimeOnly 'org.junit.jupiter:junit-jupiter-engine:5.10.0'
}

test {
    useJUnitPlatform()
}

...

---

```

3. Cấu hình triển khai Web (`web.xml` & `beans.xml`)

File `src/main/webapp/WEB-INF/beans.xml`

Tạo file này để kích hoạt CDI container:

```

<?xml
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="https://jakarta.ee/xml/ns/jakartaee"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
https://jakarta.ee/xml/ns/jakartaee/beans_4_0.xsd"

```

```
    bean-discovery-mode="all">
</beans>
```

...

File `src/main/webapp/WEB-INF/web.xml`

Cấu hình Weld Listener cho CDI và Jersey Servlet cho REST:

```
``xml
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="https://jakarta.ee/xml/ns/jakartaee"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
https://jakarta.ee/xml/ns/jakartaee/web-app_6_0.xsd"
    version="6.0">

    <!-- Listener khởi động CDI Weld trên Tomcat -->
    <listener>
        <listener-class>org.jboss.weld.environment.servlet.Listener</listener-class>
    </listener>

    <!-- Servlet JAX-RS / Jersey -->
    <servlet>
        <servlet-name>JerseyServlet</servlet-name>
        <servlet-class>org.glassfish.jersey.servlet.ServletContainer</servlet-class>
        <init-param>
            <param-name>jersey.config.server.provider.packages</param-name>
            <param-value>vn.fit.jakartaee_baitap2_tuan3.controller</param-value>
        </init-param>
        <load-on-startup>1</load-on-startup>
    </servlet>

    <servlet-mapping>
        <servlet-name>JerseyServlet</servlet-name>
        <url-pattern>/api/*</url-pattern>
    </servlet-mapping>
</web-app>
```

...

4. Xây dựng mã nguồn theo đúng cấu trúc package

Cấu trúc tuân theo ảnh đính kèm: `model`, `repository/impl`, `service`, `controller`, và `util`.

4.1. Lớp Utility

File `src/main/java/vn.fit.jakartaee\_baitap2\_tuan3/util/DBConnection.java` (giữ nguyên theo mã nguồn của bạn):

```
```java
package vn.fit.jakartaee_baitap2_tuan3.util;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DBConnection {

 private static final String URL = "jdbc:mariadb://localhost:3306/storedb";
 private static final String USER = "root";
 private static final String PASSWORD = "123456";

 private DBConnection() {
 }

 public static Connection getConnection() throws SQLException {
 try {
 Class.forName("org.mariadb.jdbc.Driver");
 } catch (ClassNotFoundException e) {
 throw new RuntimeException("Không tìm thấy MariaDB JDBC Driver", e);
 }

 return DriverManager.getConnection(URL, USER, PASSWORD);
 }
}
```
```

4.2. Package Model

File `src/main/java/vn.fit.jakartaee\_baitap2\_tuan3/model/Product.java`:

```
```java
package vn.fit.jakartaee_baitap2_tuan3.model;

public class Product {
 private int id;
 private String name;
 private double price;
 private String description;
 private String imageUrl;

 public Product() {}

 public Product(int id, String name, double price, String description, String imageUrl) {
 this.id = id;
 this.name = name;
 this.price = price;
 this.description = description;
 this.imageUrl = imageUrl;
 }

 public int getId() { return id; }
 public void setId(int id) { this.id = id; }
 public String getName() { return name; }
 public void setName(String name) { this.name = name; }
 public double getPrice() { return price; }
 public void setPrice(double price) { this.price = price; }
 public String getDescription() { return description; }
 public void setDescription(String description) { this.description = description; }
 public String getImageUrl() { return imageUrl; }
 public void setImageUrl(String imageUrl) { this.imageUrl = imageUrl; }
}

...
```
```

File `src/main/java/vn.fit.jakartaee\_baitap2\_tuan3/model/ShoppingCart.java`:

```
```java
package vn.fit.jakartaee_baitap2_tuan3.model;

import java.sql.Timestamp;
```

```

public class ShoppingCart {
 private int id;
 private int productId;
 private String customerName;
 private int quantity;
 private Timestamp createdAt;

 public ShoppingCart() {}

 public ShoppingCart(int id, int productId, String customerName, int quantity, Timestamp
createdAt) {
 this.id = id;
 this.productId = productId;
 this.customerName = customerName;
 this.quantity = quantity;
 this.createdAt = createdAt;
 }

 public int getId() { return id; }
 public void setId(int id) { this.id = id; }
 public int getProductId() { return productId; }
 public void setProductId(int productId) { this.productId = productId; }
 public String getCustomerName() { return customerName; }
 public void setCustomerName(String customerName) { this.customerName =
customerName; }
 public int getQuantity() { return quantity; }
 public void setQuantity(int quantity) { this.quantity = quantity; }
 public Timestamp getCreatedAt() { return createdAt; }
 public void setCreatedAt(Timestamp createdAt) { this.createdAt = createdAt; }
}

...

```

### ### 4.3. Package Repository & Impl

File `src/main/java/vn.fit.jakartaee\_baitap2\_tuan3/repository/ProductRepository.java`:

```

```java
package vn.fit.jakartaee_baitap2_tuan3.repository;

import vn.fit.jakartaee_baitap2_tuan3.model.Product;
import java.util.List;

```

```

public interface ProductRepository {
    List<Product> findAll();
    Product findById(int id);
    boolean save(Product product);
    boolean update(Product product);
    boolean delete(int id);
}

```

...

File

`src/main/java/vn.fit.jakartaee\_baitap2\_tuan3/repository/impl/ProductRepositoryImpl.java`:

```java

```

package vn.fit.jakartaee_baitap2_tuan3.repository.impl;

```

```

import jakarta.enterprise.context.ApplicationScoped;
import vn.fit.jakartaee_baitap2_tuan3.model.Product;
import vn.fit.jakartaee_baitap2_tuan3.repository.ProductRepository;
import vn.fit.jakartaee_baitap2_tuan3.util.DBConnection;

```

```

import java.sql.*;
import java.util.ArrayList;
import java.util.List;

```

@ApplicationScoped

```

public class ProductRepositoryImpl implements ProductRepository {

```

@Override

```

public List<Product> findAll() {
 List<Product> list = new ArrayList<>();
 String sql = "SELECT * FROM Product";
 try (Connection conn = DBConnection.getConnection();
 PreparedStatement ps = conn.prepareStatement(sql);
 ResultSet rs = ps.executeQuery()) {
 while (rs.next()) {
 list.add(new Product(
 rs.getInt("id"),
 rs.getString("name"),
 rs.getDouble("price"),
 rs.getString("description"),
 rs.getString("image_url")
));
 }
 } catch (SQLException e) {
 e.printStackTrace();
 }
}

```

```
 return list;
}
```

@Override

```
public Product findById(int id) {
 String sql = "SELECT * FROM Product WHERE id = ?";
 try (Connection conn = DBConnection.getConnection();
 PreparedStatement ps = conn.prepareStatement(sql)) {
 ps.setInt(1, id);
 try (ResultSet rs = ps.executeQuery()) {
 if (rs.next()) {
 return new Product(
 rs.getInt("id"),
 rs.getString("name"),
 rs.getDouble("price"),
 rs.getString("description"),
 rs.getString("image_url")
);
 }
 }
 } catch (SQLException e) {
 e.printStackTrace();
 }
 return null;
}
```

@Override

```
public boolean save(Product product) {
 String sql = "INSERT INTO Product (name, price, description, image_url) VALUES (?, ?, ?, ?)";
 try (Connection conn = DBConnection.getConnection();
 PreparedStatement ps = conn.prepareStatement(sql)) {
 ps.setString(1, product.getName());
 ps.setDouble(2, product.getPrice());
 ps.setString(3, product.getDescription());
 ps.setString(4, product.getImageUrl());
 return ps.executeUpdate() > 0;
 } catch (SQLException e) {
 e.printStackTrace();
 return false;
 }
}
```

@Override

```
public boolean update(Product product) {
 String sql = "UPDATE Product SET name = ?, price = ?, description = ?, image_url = ? WHERE id = ?";
 try (Connection conn = DBConnection.getConnection());
```

```

 PreparedStatement ps = conn.prepareStatement(sql)) {
 ps.setString(1, product.getName());
 ps.setDouble(2, product.getPrice());
 ps.setString(3, product.getDescription());
 ps.setString(4, product.getImageUrl());
 ps.setInt(5, product.getId());
 return ps.executeUpdate() > 0;
} catch (SQLException e) {
 e.printStackTrace();
 return false;
}
}

```

```

@Override
public boolean delete(int id) {
 String sql = "DELETE FROM Product WHERE id = ?";
 try (Connection conn = DBConnection.getConnection();
 PreparedStatement ps = conn.prepareStatement(sql)) {
 ps.setInt(1, id);
 return ps.executeUpdate() > 0;
 } catch (SQLException e) {
 e.printStackTrace();
 return false;
 }
}
}
...

```

File `src/main/java/vn.fit.jakartaee\_baitap2\_tuan3/repository/ShoppingCartRepository.java`:

```

```java
package vn.fit.jakartaee_baitap2_tuan3.repository;

import vn.fit.jakartaee_baitap2_tuan3.model.ShoppingCart;
import java.util.List;

public interface ShoppingCartRepository {
    List<ShoppingCart> findAll();
    ShoppingCart findById(int id);
    boolean save(ShoppingCart cart);
    boolean update(ShoppingCart cart);
    boolean delete(int id);
}
...

```

File

`src/main/java/vn.fit.jakartaee\_baitap2\_tuan3/repository/impl/ShoppingCartRepositoryImpl.java`:

```
```java
package vn.fit.jakartaee_baitap2_tuan3.repository.impl;

import jakarta.enterprise.context.ApplicationScoped;
import vn.fit.jakartaee_baitap2_tuan3.model.ShoppingCart;
import vn.fit.jakartaee_baitap2_tuan3.repository.ShoppingCartRepository;
import vn.fit.jakartaee_baitap2_tuan3.util.DBConnection;

import java.sql.*;
import java.util.ArrayList;
import java.util.List;

@ApplicationScoped
public class ShoppingCartRepositoryImpl implements ShoppingCartRepository {

 //Sinh viên tự thực hiện
}

...

```

## ### 4.4. Package Service

File `src/main/java/vn.fit.jakartaee\_baitap2\_tuan3/service/ProductService.java`:

```
```java
package vn.fit.jakartaee_baitap2_tuan3.service;

import jakarta.enterprise.context.ApplicationScoped;
import jakarta.inject.Inject;
import vn.fit.jakartaee_baitap2_tuan3.model.Product;
import vn.fit.jakartaee_baitap2_tuan3.repository.ProductRepository;

import java.util.List;

@ApplicationScoped
public class ProductService {

    @Inject
```

```

private ProductRepository productRepository;

public List<Product> getAllProducts() {
    return productRepository.findAll();
}

public Product getProductById(int id) {
    return productRepository.findById(id);
}

public boolean createProduct(Product product) {
    return productRepository.save(product);
}

public boolean updateProduct(Product product) {
    return productRepository.update(product);
}

public boolean deleteProduct(int id) {
    return productRepository.delete(id);
}
}
...

```

File `src/main/java/vn.fit.jakartaee\_baitap2\_tuan3/service/ShoppingCartService.java`:

```

```java
package vn.fit.jakartaee_baitap2_tuan3.service;

import jakarta.enterprise.context.ApplicationScoped;
import jakarta.inject.Inject;
import vn.fit.jakartaee_baitap2_tuan3.model.ShoppingCart;
import vn.fit.jakartaee_baitap2_tuan3.repository.ShoppingCartRepository;

import java.util.List;

@ApplicationScoped
public class ShoppingCartService {

 @Inject
 private ShoppingCartRepository cartRepository;

 public List<ShoppingCart> getAllCarts() {
 return cartRepository.findAll();
 }

 public ShoppingCart getCartById(int id) {

```

```

 return cartRepository.findById(id);
 }

 public boolean createCart(ShoppingCart cart) {
 return cartRepository.save(cart);
 }

 public boolean updateCart(ShoppingCart cart) {
 return cartRepository.update(cart);
 }

 public boolean deleteCart(int id) {
 return cartRepository.delete(id);
 }
}

...

```

## ### 4.5. Package Controller (JAX-RS REST Endpoints)

File `src/main/java/vn.fit.jakartaee\_baitap2\_tuan3/controller/ProductController.java`:

```

```java
package vn.fit.jakartaee_baitap2_tuan3.controller;

import jakarta.enterprise.context.RequestScoped;
import jakarta.inject.Inject;
import jakarta.ws.rs.*;
import jakarta.ws.rs.core.MediaType;
import jakarta.ws.rs.core.Response;
import vn.fit.jakartaee_baitap2_tuan3.model.Product;
import vn.fit.jakartaee_baitap2_tuan3.service.ProductService;

import java.util.List;

@RequestScoped
@Path("/products")
@Produces(MediaType.APPLICATION_JSON)
@Consumes(MediaType.APPLICATION_JSON)
public class ProductController {

    @Inject
```

```
private ProductService productService;
```

```
@GET
```

```
public Response getAll() {  
    List<Product> products = productService.getAllProducts();  
    return Response.ok(products).build();  
}
```

```
@GET
```

```
@Path("/{id}")
```

```
public Response getById(@PathParam("id") int id) {  
    Product p = productService.getProductById(id);  
    if (p == null) {  
        return Response.status(Response.Status.NOT_FOUND).entity("{\"message\":  
        \"Product not found\"}").build();  
    }  
    return Response.ok(p).build();  
}
```

```
@POST
```

```
public Response create(Product product) {  
    boolean success = productService.createProduct(product);  
    if (success) {  
        return Response.status(Response.Status.CREATED).entity("{\"message\": \"Product  
        created\"}").build();  
    }  
    return Response.status(Response.Status.BAD_REQUEST).entity("{\"message\":  
    \"Failed to create product\"}").build();  
}
```

```
@PUT
```

```
@Path("/{id}")
```

```
public Response update(@PathParam("id") int id, Product product) {  
    product.setId(id);  
    boolean success = productService.updateProduct(product);  
    if (success) {  
        return Response.ok("{\"message\": \"Product updated\"}").build();  
    }  
    return Response.status(Response.Status.NOT_MODIFIED).build();  
}
```

```
@DELETE
```

```
@Path("/{id}")
```

```
public Response delete(@PathParam("id") int id) {  
    boolean success = productService.deleteProduct(id);  
    if (success) {  
        return Response.ok("{\"message\": \"Product deleted\"}").build();  
    }  
}
```

```

        return Response.status(Response.Status.NOT_FOUND).build();
    }
}
...

```

File `src/main/java/vn.fit.jakartaee\_baitap2\_tuan3/controller/ShoppingCartController.java`:

```

```java
package vn.fit.jakartaee_baitap2_tuan3.controller;

import jakarta.enterprise.context.RequestScoped;
import jakarta.inject.Inject;
import jakarta.ws.rs.*;
import jakarta.ws.rs.core.MediaType;
import jakarta.ws.rs.core.Response;
import vn.fit.jakartaee_baitap2_tuan3.model.ShoppingCart;
import vn.fit.jakartaee_baitap2_tuan3.service.ShoppingCartService;

import java.util.List;

@RequestScoped
@Path("/carts")
@Produces(MediaType.APPLICATION_JSON)
@Consumes(MediaType.APPLICATION_JSON)
public class ShoppingCartController {

 @Inject
 private ShoppingCartService cartService;

 @GET
 public Response getAll() {
 return Response.ok(cartService.getAllCarts()).build();
 }

 @GET
 @Path("/{id}")
 public Response getById(@PathParam("id") int id) {
 ShoppingCart cart = cartService.getCartById(id);
 if (cart == null) {
 return Response.status(Response.Status.NOT_FOUND).entity("{\"message\": \"Cart item not found\"}").build();
 }
 return Response.ok(cart).build();
 }

 @POST
 public Response create(ShoppingCart cart) {

```

```

 boolean success = cartService.createCart(cart);
 if (success) {
 return Response.status(Response.Status.CREATED).entity("{\"message\": \"Cart item created\"}").build();
 }
 return Response.status(Response.Status.BAD_REQUEST).entity("{\"message\": \"Failed to create cart item\"}").build();
 }

 @PUT
 @Path("/{id}")
 public Response update(@PathParam("id") int id, ShoppingCart cart) {
 cart.setId(id);
 boolean success = cartService.updateCart(cart);
 if (success) {
 return Response.ok("{\"message\": \"Cart item updated\"}").build();
 }
 return Response.status(Response.Status.NOT_MODIFIED).build();
 }

 @DELETE
 @Path("/{id}")
 public Response delete(@PathParam("id") int id) {
 boolean success = cartService.deleteCart(id);
 if (success) {
 return Response.ok("{\"message\": \"Cart item deleted\"}").build();
 }
 return Response.status(Response.Status.NOT_FOUND).build();
 }
}

...

```

## ## 5. Triển khai ứng dụng trên IntelliJ IDEA & Tomcat 10

1. **Kiểm tra JDK**: Vào **File** > **Project Structure** > **Project**, đảm bảo SDK được đặt là **JDK 17**.
2. **Cấu hình Run/Debug Configuration**:
  - \* Nhấn **Run** > **Edit Configurations...** > Thêm dấu '+' > chọn **Tomcat Server** > **Local**.
  - \* Trỏ 'Tomcat Home' đến thư mục cài đặt **Apache Tomcat 10.x**.

\* Chuyển sang tab **Deployment** > Nhấn dấu `+` > chọn **Artifact...** > chọn artifact dạng `...:war exploded`.  
\* Ở mục **Application context**, đặt đường dẫn là `/` hoặc tên project (ví dụ: `/jakartae\_baitap2\_tuan3`). Giả định để context path là: `/api-app`.

3. Nhấn **Apply** > **Run** để khởi chạy Tomcat.

---

## ## 6. Hướng dẫn Test các API bằng Postman

Giả sử ứng dụng chạy tại: `http://localhost:8080/api-app/api`

### ### 1. Test Product API

\* **GET All Products**:

\* Method: `GET`

\* URL: `http://localhost:8080/api-app/api/products`

\* **GET Product by ID**:

\* Method: `GET`

\* URL: `http://localhost:8080/api-app/api/products/1`

\* **POST Create Product**:

\* Method: `POST`

\* URL: `http://localhost:8080/api-app/api/products`

\* Header: `Content-Type: application/json`

\* Body (raw JSON):

```
```json
{
  "name": "AirPods Pro 2",
  "price": 249.0,
  "description": "Active Noise Cancellation",
  "imageUrl": "https://picsum.photos/id/11/400/400"
}
```

...

* **PUT Update Product**:

* Method: `PUT`

* URL: `http://localhost:8080/api-app/api/products/1`

* Header: `Content-Type: application/json`

```
* Body (raw JSON):
```json
{
 "name": "iPhone 15 Pro Max",
 "price": 1199.0,
 "description": "Updated description",
 "imageUrl": "https://picsum.photos/id/1/400/400"
}

...

```

```
* **DELETE Product**:
* Method: `DELETE`
* URL: `http://localhost:8080/api-app/api/products/1`

```

## ### 2. Test ShoppingCart API

```
* **GET All Carts**:
* Method: `GET`
* URL: `http://localhost:8080/api-app/api/carts`

```

```
* **GET Cart by ID**:
* Method: `GET`
* URL: `http://localhost:8080/api-app/api/carts/2`

```

```
* **POST Create Cart**:
* Method: `POST`
* URL: `http://localhost:8080/api-app/api/carts`
* Header: `Content-Type: application/json`
* Body (raw JSON):
```json
{
  "productId": 2,
  "customerName": "Vu Thi Mai",
  "quantity": 3
}

...

```

```
* **PUT Update Cart**:  
* Method: `PUT`  
* URL: `http://localhost:8080/api-app/api/carts/2`  
* Header: `Content-Type: application/json`  
* Body (raw JSON):  
``json  
{  
  "productId": 2,  
  "customerName": "Tran Thi B",  
  "quantity": 5  
}  
...`
```

```
* **DELETE Cart Item**:  
* Method: `DELETE`  
* URL: `
```

Nghịệp vụ thực tế yêu cầu tính toán logic, kết hợp dữ liệu giữa cả 2 bảng `Product` và `ShoppingCart`, kèm hướng dẫn chi tiết về luồng xử lý ở tầng `Service`, `Controller`, cách kiểm thử trên **Postman** và câu lệnh SQL chứng minh trên **Database**.

Yêu cầu 1: Tính tổng hóa đơn giỏ hàng của một khách hàng (Customer Cart Bill Breakdown)

- **Mục tiêu & Nghiệp vụ:** Tính tổng giá tiền thanh toán của một khách hàng theo `customerName`. Service cần lấy danh sách các mục trong giỏ hàng của khách đó, lấy thông tin đơn giá từ bảng `Product`, tính thành tiền từng món ($\text{quantity} * \text{price}$) và tính tổng bill (`grandTotal`), đồng thời áp dụng chính sách giảm 10% nếu tổng bill trên \$2,000.
- **Tác động 2 bảng:** Đọc bảng `ShoppingCart` theo tên khách và đối chiếu đơn giá từ bảng `Product`.

1. Triển khai Service & Controller

- **Service** (ShoppingCartService):
-

- **Java**

```
public CustomerBillDTO calculateCustomerBill(String customerName) {  
    // 1. Lấy tất cả items của customerName từ ShoppingCartRepository  
    // 2. Với mỗi item, gọi ProductRepository.findById(productId) để lấy giá  
    // 3. Tính: itemTotal = quantity * price  
    // 4. Cộng dồn subTotal, nếu subTotal > 2000 thì discount = subTotal * 0.1, finalTotal = subTotal  
    - discount  
    // 5. Trả về đối tượng DTO chứa chi tiết và tổng tiền  
}
```

-
- **Controller** (ShoppingCartController):

Java

```
@GET  
@Path("/bill/{customerName}")  
public Response getBillByCustomer(@PathParam("customerName") String customerName) {  
    CustomerBillDTO bill = cartService.calculateCustomerBill(customerName);  
    return Response.ok(bill).build();  
}
```

2. Kiểm thử trên Postman

- **Request:** GET http://localhost:8080/api-app/api/carts/bill/Nguyen%20Van%20A
-
- **Response mong đợi (200 OK):**
-

- **JSON**

```
{  
  "customerName": "Nguyen Van A",  
  "items": [  
    {  
      "cartId": 1,  
      "productName": "iPhone 15 Pro",  
      "price": 999.0,  
      "quantity": 1,  
      "subTotal": 999.0  
    }  
  ],  
  "subTotal": 999.0,  
  "discount": 0.0,  
  "finalTotal": 999.0  
}
```

3. Chứng minh kết quả trên Database

Chạy query SQL để đối chiếu kết quả trả về từ Postman:

SQL

```
SELECT
  c.customer_name,
  p.name AS product_name,
  p.price,
  c.quantity,
  (p.price * c.quantity) AS item_total
FROM ShoppingCart c
JOIN Product p ON c.product_id = p.id
WHERE c.customer_name = 'Nguyen Van A';
```

Yêu cầu 2: Thêm sản phẩm vào giỏ hàng có kiểm tra ràng buộc giá trị tối đa (Add Item with Value Cap)

- **Mục tiêu & Nghiệp vụ:** Khi thêm 1 sản phẩm vào giỏ hàng, hệ thống kiểm tra sản phẩm có tồn tại hay không. Đồng thời tính giá trị lô hàng thêm vào ($\text{product.price} * \text{quantity}$). Nếu giá trị vượt quá \$5,000, từ chối thêm để tránh rủi ro gian lận. Nếu hợp lệ, lưu vào DB và trả về thông tin item kèm tổng giá trị tính toán được.
- **Tác động 2 bảng:** Truy vấn Product để xác thực ID + lấy đơn giá, tính toán điều kiện, sau đó ghi dữ liệu vào ShoppingCart.

1. Triển khai Service & Controller

- **Service** (ShoppingCartService):

- **Java**

```
public AddCartResponseDTO addItemToCart(ShoppingCart cart) {
    Product product = productRepository.findById(cart.getProductid());
    if (product == null) {
        throw new IllegalArgumentException("Sản phẩm không tồn tại!");
    }
    double estimatedTotal = product.getPrice() * cart.getQuantity();
    if (estimatedTotal > 5000.0) {
        throw new IllegalStateException("Giá trị đơn hàng vượt quá hạn mức cho phép ($5,000)!");
    }
    cartRepository.save(cart);
    return new AddCartResponseDTO(cart.getId(), product.getName(), cart.getQuantity(),
    estimatedTotal);
}
```

- **Controller** (ShoppingCartController):

- Java

@POST

@Path("/add-with-cap")

public Response addWithCap(ShoppingCart cart) {

try {

AddCartResponseDTO result = cartService.addItemToCart(cart);

return Response.status(Response.Status.CREATED).entity(result).build();

} catch (IllegalStateException | IllegalArgumentException e) {

return Response.status(Response.Status.BAD_REQUEST).entity("{\"error\": \"" +

e.getMessage() + "\"}").build();

}

}

-

2. Kiểm thử trên Postman

- Test Case Hợp lệ:

-

- Method: POST

-

- URL: http://localhost:8080/api-app/api/carts/add-with-cap

-

- Body:

-

- JSON

{

"productId": 3,

"customerName": "Sinh Vien Test",

"quantity": 2

}

-

- Response: 201 Created kèm estimatedTotal = 2398.0 (MacBook Air \$1199 * 2).

-

- Test Case Bị Chặn:

-

- Body: {"productId": 3, "customerName": "Sinh Vien Test", "quantity": 10} (1199 * 10 = \$11,990 > \$5,000).

-

- Response: 400 Bad Request với message lỗi.

-

3. Chứng minh kết quả trên Database

Xác nhận bản ghi mới đã được lưu vào bảng ShoppingCart:

SQL

```
SELECT c.*, p.name, p.price, (c.quantity * p.price) AS total_amount
FROM ShoppingCart c
JOIN Product p ON c.product_id = p.id
WHERE c.customer_name = 'Sinh Vien Test';
```

Yêu cầu 3: Điều chỉnh giá sản phẩm hàng loạt theo doanh số giỏ hàng (Dynamic Repricing by Popularity)

- **Mục tiêu & Nhiệm vụ:** Tính tổng số lượng đặt (`SUM(quantity)`) của từng sản phẩm trong toàn bộ ShoppingCart.
- - Nếu sản phẩm có tổng lượt đặt `>= 5`, tăng giá sản phẩm đó thêm 10% (hot item).
 -
 - Nếu sản phẩm chưa có lượt đặt nào (`quantity = 0`), giảm giá 5% để kích cầu.
 -
 - Cập nhật lại giá mới vào bảng Product và trả về danh sách biến động giá.
 -
- **Tác động 2 bảng:** Tính toán thống kê từ ShoppingCart, sau đó cập nhật (UPDATE) trực tiếp dữ liệu ở bảng Product.

1. Triển khai Service & Controller

- **Service (ProductService):**

- Java

```
public List<RepriceReportDTO> applyDynamicRepricing() {
    // 1. Truy vấn danh sách Product và map số lượng từ ShoppingCart
    // 2. Tính newPrice theo công thức:
    //    - If totalQty >= 5: newPrice = oldPrice * 1.10
    //    - If totalQty == 0: newPrice = oldPrice * 0.95
    // 3. Gọi productRepository.update(product) cập nhật giá mới vào DB
    // 4. Trả về báo cáo gồm: productId, name, oldPrice, newPrice, totalOrdered
}
```

- **Controller (ProductController):**

- Java

```
@PUT
@Path("/dynamic-pricing")
public Response dynamicPricing() {
    List<RepriceReportDTO> report = productService.applyDynamicRepricing();
    return Response.ok(report).build();
}
```

2. Kiểm thử trên Postman

- Request: PUT http://localhost:8080/api-app/api/products/dynamic-pricing
-
- Response (200 OK):
-

- JSON

```
[
{
  "productId": 2,
  "productName": "Samsung Galaxy S24",
  "totalQuantityOrdered": 6,
  "oldPrice": 850.0,
  "newPrice": 935.0,
  "adjustment": "+10%"
}
]
```

-

3. Chứng minh kết quả trên Database

- Trước và sau khi gọi API, sinh viên chạy query đối chiếu số lượng đã đặt và giá được cập nhật:

-

```
SQL
SELECT
  p.id,
  p.name,
  p.price AS current_price,
  COALESCE(SUM(c.quantity), 0) AS total_ordered
FROM Product p
LEFT JOIN ShoppingCart c ON p.id = c.product_id
GROUP BY p.id, p.name, p.price;
```

Yêu cầu 4: Chuyển đổi toàn bộ giỏ hàng của khách thành Đơn đặt mua và Xóa giỏ (Checkout & Clear Cart)

- Mục tiêu & Nhiệm vụ: Mô phỏng hành động "Thanh toán" (Checkout).
- - Nhận `customerName`.
 -
 - Tính tổng số lượng món hàng, tổng tiền cần thanh toán dựa trên các items trong `ShoppingCart` và đơn giá tương ứng trong `Product`.
 -
 - Sau khi hoàn tất tính toán tổng kết, tiến hành **xóa sạch** các bản ghi giỏ hàng của khách hàng này trong bảng `ShoppingCart` (đánh dấu đã checkout xong).
 -
- Tác động 2 bảng: Tính toán kết hợp `Product` & `ShoppingCart`, sau đó xóa các bản ghi thuộc `ShoppingCart`.

-
- 1. Triển khai Service & Controller
- Service (ShoppingCartService):
-

- Java

```
public CheckoutSummaryDTO checkout(String customerName) {
    // 1. Lấy danh sách ShoppingCart theo customerName
    // 2. Tính tổng số mặt hàng (itemsCount), tổng số lượng (totalQuantity), tổng tiền (totalAmount) từ Product.price
    // 3. Xóa các record của customerName trong ShoppingCart: DELETE FROM ShoppingCart WHERE customer_name = ?
    // 4. Trả về phiếu tóm tắt thanh toán CheckoutSummaryDTO
}
```

-
- Controller (ShoppingCartController):
-

- Java

```
@POST
@Path("/checkout/{customerName}")
public Response checkout(@PathParam("customerName") String customerName) {
    CheckoutSummaryDTO summary = cartService.checkout(customerName);
    return Response.ok(summary).build();
}
```

2. Kiểm thử trên Postman

- Request: POST http://localhost:8080/api-app/api/carts/checkout/Le%20Van%20C
-
- Response (200 OK):
-

- JSON

```
{
  "customerName": "Le Van C",
  "totalItems": 1,
  "totalQuantity": 1,
  "totalPaid": 1199.0,
  "status": "CHECKOUT_COMPLETED_AND_CART_CLEARED"
}
```

3. Chứng minh kết quả trên Database

- Kiểm tra bảng ShoppingCart xác nhận không còn dữ liệu của khách hàng vừa checkout:
-

SQL

```
SELECT * FROM ShoppingCart WHERE customer_name = 'Le Van C';  
-- Kết quả trả về: 0 rows (đã bị xóa thành công)
```

Yêu cầu 5: Thông kê Doanh thu dự kiến & Đóng góp % từng danh mục sản phẩm (Revenue Share Analytics)

- **Mục tiêu & Nghiệp vụ:** Phân tích dữ liệu kinh doanh của toàn sản.
- - Tính doanh thu tiềm năng của từng sản phẩm trong giỏ hàng:
 $\text{productRevenue} = \text{p.price} * \text{SUM}(\text{c.quantity})$.
 -
 - Tính tổng doanh thu toàn bộ giỏ hàng: $\text{totalRevenue} = \text{SUM}(\text{productRevenue})$.
 -
 - Tính tỷ lệ đóng góp doanh số theo phần trăm của từng sản phẩm:
 $\text{contributionPercentage} = (\text{productRevenue} / \text{totalRevenue}) * 100$.
 -
- **Tác động 2 bảng:** Thực hiện kết nối (JOIN) Product và ShoppingCart, gom nhóm (GROUP BY) và tính toán tỷ lệ thống kê.

1. Triển khai Service & Controller

- **Service** (ProductService hoặc tạo một AnalyticsService):
-

- Java

```
public List<RevenueShareDTO> getRevenueShareReport() {  
    // 1. Tính tổng doanh thu của tất cả mặt hàng có trong ShoppingCart  
    // 2. Với mỗi sản phẩm có trong giỏ hàng, tính tổng doanh thu riêng của nó  
    // 3. Tính tỷ lệ %: (productRevenue / totalRevenue) * 100 (làm tròn 2 chữ số thập phân)  
    // 4. Sắp xếp giảm dần theo doanh thu  
}
```

-
- **Controller** (ProductController):
-

- Java

```
@GET  
@Path("/analytics/revenue-share")  
public Response getRevenueShare() {  
    List<RevenueShareDTO> stats = productService.getRevenueShareReport();  
    return Response.ok(stats).build();  
}
```

2. Kiểm thử trên Postman

- **Request:** GET <http://localhost:8080/api-app/api/products/analytics/revenue-share>

-
- **Response (200 OK):**
-

- **JSON**

```
[
  {
    "productId": 3,
    "productName": "MacBook Air M2",
    "totalUnitsInCart": 2,
    "revenueContribution": 2398.0,
    "percentage": "22.45%"
  },
  {
    "productId": 1,
    "productName": "iPhone 15 Pro",
    "totalUnitsInCart": 3,
    "revenueContribution": 2997.0,
    "percentage": "28.05%"
  }
]
```

3. Chứng minh kết quả trên Database

Sử dụng hàm cửa sổ (Window Function) trong MariaDB để kiểm chứng:

SQL

```
SELECT
  p.id,
  p.name,
  SUM(c.quantity) AS total_units,
  (p.price * SUM(c.quantity)) AS revenue,
  ROUND(
    (p.price * SUM(c.quantity)) /
    SUM(p.price * SUM(c.quantity)) OVER () * 100,
    2
  ) AS contribution_percentage
FROM Product p
JOIN ShoppingCart c ON p.id = c.product_id
GROUP BY p.id, p.name, p.price
ORDER BY revenue DESC;
```

Tiêu chí chấm điểm / Báo cáo dành cho sinh viên:

1. **CDI Injection:** @Inject đúng Service vào Controller và @Inject Repository vào Service.
- 2.
3. **Logic tính toán:** Đúng các điều kiện logic (chiết khấu, giới hạn giá trị đơn, cập nhật giá, tính % doanh số).
- 4.

5. **Ảnh chụp Postman:** Minh chứng Request URL, HTTP Method, Request Body (nếu có), HTTP Status Code và Response JSON.
- 6.
7. **Ảnh chụp Database:** Chụp màn hình bảng trước và sau thao tác (đặc biệt với các API làm thay đổi dữ liệu như Yêu cầu 2, 3, 4) hoặc bảng kết quả chạy SQL Query đối chiếu.